Design and Verification of a SECDED-Protected Synchronous FIFO with Programmable Thresholds and Built-in Fault Injection

Computational Project Report
M.Tech - VLSI Design (Digital)

Submitted by: _____ Roll No.: _____

Guide: _____

Department: _____

Tools: Verilog-2005 RTL, Icarus Verilog 12.0, Verilator 5.020, Yosys 0.70 + ABC, Python 3 (matplotlib, reportlab)

October 2026

Abstract

First-in first-out (FIFO) buffers sit between almost every pair of pipeline stages in a digital system, and in deep-submicron technologies the storage array of a FIFO is exposed to radiation- and noise-induced single-event upsets. A conventional synchronous FIFO forwards a flipped bit silently. This project designs, verifies and synthesises a parameterised single-clock FIFO in Verilog that protects its storage with an extended-Hamming single-error-correcting, double-error-detecting (SECDED) code, offers run-time programmable almost-full / almost-empty thresholds, registered glitch-free status flags, write-through when full, sticky overflow / underflow flags, saturating error counters, and a fault-injection port that makes the protection testable. The design is verified with a cycle-accurate scoreboard testbench that checks every output on every clock: the main run simulates 116489 cycles with 1486037 comparisons and 0 mismatches, all 39 codeword bit positions are fault-injected, and a 14-run regression across widths 8-64 and depths 4-64 passes. The strength of the testbench itself is demonstrated by mutation testing: 15 of 15 deliberately injected design bugs were detected. Synthesis with Yosys/ABC shows that the SECDED overhead falls from 108.8 % at DEPTH=8 to 32.9 % at DEPTH=64 as the fixed encoder/decoder cost is amortised, while an analytical model shows a ~43,000-fold reduction of silent data errors at a per-bit upset probability of 10^{-6}.

Keywords: synchronous FIFO, SECDED, Hamming code, soft error, fault injection, self-checking testbench, mutation testing, Verilog, Yosys.

1. Introduction

A synchronous FIFO is a dual-ported memory with a write pointer and a read pointer driven by the same clock. It decouples a producer from a consumer that do not always operate at the same rate, and appears in bus bridges, network-on-chip routers, DSP data paths, DMA engines and UART/SPI controllers. Because the storage array is usually the largest block of the FIFO, it is also the part most likely to suffer a soft error - a transient bit flip caused by an energetic particle or supply noise. With shrinking node capacitance the critical charge needed to flip a cell keeps falling, so error-correcting codes, long standard in caches and DRAM, are increasingly applied to small buffers in safety-relevant designs (automotive, avionics, space).

The goal of this project is not only to write a working FIFO, but to produce a *measurably* more robust one and to prove its correctness with a verification flow that can itself be shown to be effective. All results in this report were produced by the scripts included with the project and can be reproduced with a single command (`bash scripts/run_all.sh`).

1.1 Objectives

(1) Design a fully parameterised synchronous FIFO (data width, depth, ECC on/off) in synthesisable Verilog-2005. (2) Protect the storage with a generic SECDED code whose check-bit count is derived automatically from the data width. (3) Add the control and observability features that a practical IP block needs. (4) Verify the design with a self-checking reference-model testbench and functional coverage, and measure the testbench's bug-finding ability by mutation testing. (5) Quantify the hardware cost by logic synthesis and compare with a conventional FIFO.

1.2 Contribution and positioning

ECC-protected FIFOs exist in commercial FPGA/ASIC IP generators, where the ECC is typically provided by the vendor's block-RAM primitive and is not visible as RTL. The contribution of this project is an open, vendor-independent, single-file-per-block RTL implementation that combines: a width-generic extended-Hamming codec written with elaboration-time loops (no hand-written parity equations), flags computed from the *next* occupancy and registered, write-through-when-full for full throughput, a codeword-level fault-injection port for verification and built-in self-test, and a quantitative evaluation (coverage, mutation score, area/depth scaling and a reliability model). The project does not claim a new coding scheme; its novelty lies in this integration and in the evidence-based evaluation.

2. Background

2.1 Conventional synchronous FIFO

The classic design uses (N+1)-bit read and write pointers for a 2^N-deep array. The FIFO is empty when the pointers are equal and full when they differ only in the MSB [1]. Writes when full and reads when empty must be blocked. In this classic form the full/empty flags are combinational comparisons of the pointers and there is no indication when data is corrupted inside the array. This design is implemented as `fifo_baseline.v` and used as the reference for comparison.

2.2 Extended Hamming (SECDED) code

A Hamming code places P check bits at the power-of-two positions 1, 2, 4, ... of a codeword and data bits in the remaining positions [2]. Check bit j is the XOR of every position whose index has bit j set. For K data bits, P is the smallest integer with $2^P \geq K + P + 1$. On reading, recomputing the checks gives a P-bit syndrome equal to the index of a single flipped bit. Adding one overall parity bit (extended Hamming) gives SECDED [2],[3]: if the overall parity fails, a single error occurred and the syndrome locates it (syndrome 0 means the parity bit itself flipped); if the overall parity passes but the syndrome is non-zero, two bits flipped and the word is flagged as uncorrectable. For K = 32 this gives the (39,32) code, i.e. 7 check bits (21.9 % storage overhead).

3. Proposed Architecture

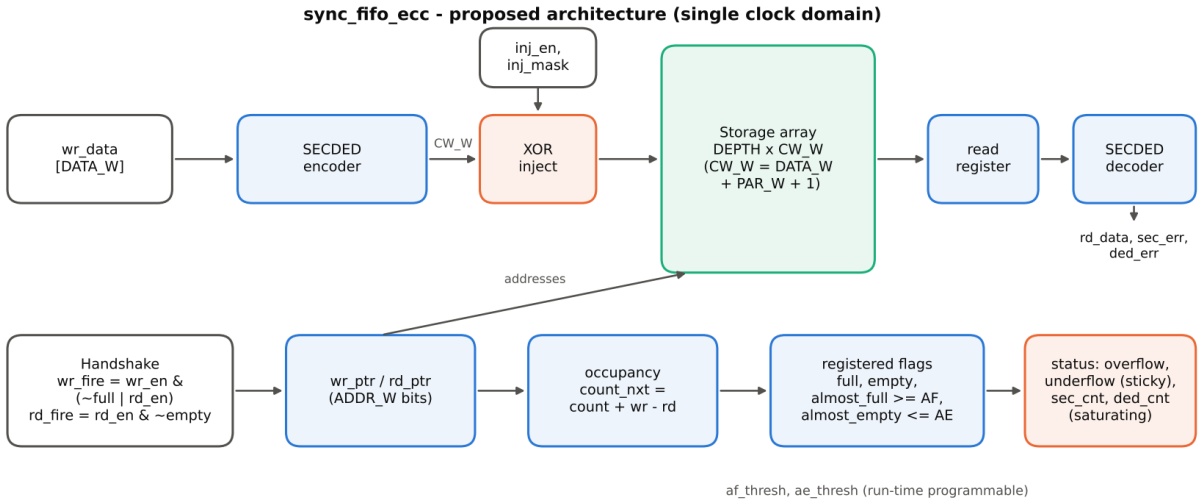


Figure 1 - Block diagram of the proposed sync_fifo_ecc.

Write data is encoded into a CW_W-bit codeword, optionally XOR-ed with a fault mask, and written into the array at `wr_ptr`. A read captures the codeword at `rd_ptr` into a read register; the decoder after this register corrects single errors and raises `sec_err` or `ded_err`, both qualified by `rd_valid`. The control path keeps an occupancy counter and computes every flag from the next value of the counter, so all flags leave the block from flip-flops (glitch-free, no pointer comparator in the output path).

3.1 Parameters

Parameter	Default	Meaning
DATA_W	32	data width (any value; check bits derived automatically)
DEPTH	16	number of words, power of two
ECC_EN	1	1 = SECDED storage, 0 = plain storage (same control logic)
PAR_W	derived	$clog2(DATA_W + $clog2(DATA_W) + 1), verified minimal by tb_secded
CW_W	derived	DATA_W + PAR_W + 1 (39 for DATA_W = 32)

3.2 Interface

Port	Dir	Description
clk, rst_n	in	clock; asynchronous active-low reset
wr_en, wr_data	in	write request and data
rd_en	in	read request
rd_data, rd_valid	out	corrected read data, valid one clock after an accepted read
full, empty, count	out	registered status and occupancy
af_thresh, ae_thresh	in	run-time thresholds
almost_full, almost_empty	out	count ≥ af_thresh, count ≤ ae_thresh (registered)
overflow, underflow	out	sticky: a write was dropped / a read was attempted when empty
sec_err, ded_err	out	single error corrected / double error detected on this read

Port	Dir	Description
sec_cnt, ded_cnt	out	16-bit saturating event counters
clr_status	in	synchronous clear of sticky flags and counters
inj_en, inj_mask	in	fault injection: XOR mask applied to the codeword being written

4. Implementation

4.1 Generic SECDED codec

Both `secded_enc.v` and `secded_dec.v` are written with `for` loops over codeword positions inside combinational always blocks. The loops are unrolled at elaboration, so the same source produces the (13,8), (22,16), (39,32) and (72,64) codes without any hand-derived equations. Codeword bit 0 holds the overall parity; positions 1..HAM_W hold the Hamming code. The decoder computes the syndrome and the overall parity, flips the bit addressed by the syndrome when a single error is indicated, and reports a syndrome that points beyond the codeword (only possible with three or more errors) as uncorrectable.

4.2 Handshake and write-through

An accepted read is `rd_fire = rd_en & ~empty`. An accepted write is `wr_fire = wr_en & (~full | rd_en)`: when the FIFO is full and a read occurs in the same clock, the write is accepted too, because the read register captures the old word at `rd_ptr` (equal to `wr_ptr` when full) on the same edge as the new word is written. A full FIFO therefore streams at one word per clock instead of stalling every other cycle. Because the occupancy counter carries the full/empty information, the pointers need only ADDR_W bits.

4.3 Registered flags

With `count_nxt = count + wr_fire - rd_fire`, every flag is registered as a function of `count_nxt` (e.g. `full <= (count_nxt == DEPTH)`). The flags therefore change on the same edge as the occupancy, are glitch-free, and the thresholds can be changed at any time; the new comparison takes effect on the next clock.

4.4 Error reporting and fault injection

Sticky overflow/underflow flags record protocol violations by the surrounding logic. SEC/DED events increment 16-bit counters that saturate at 65535 instead of wrapping, so a high error count can never be mistaken for a low one. A synchronous `clr_status` has priority over new events. The `inj_mask` port XORs an arbitrary pattern into the stored codeword; it is used by the testbench to reach every bit position and can be used in silicon as a built-in self-test of the ECC path. Tied to zero, it is removed by synthesis.

4.5 Source files

File	Lines	Content
rtl/sync_fifo_ecc.v	175	proposed FIFO (top level)
rtl/secded_enc.v	54	generic SECDED encoder
rtl/secded_dec.v	72	generic SECDED decoder
rtl/fifo_baseline.v	55	conventional FIFO for comparison
tb/tb_sync_fifo_ecc.v	414	scoreboard testbench, coverage, fault injection
tb/tb_secded.v	102	exhaustive/random codec unit test
tb/tb_fifo_baseline.v	63	testbench of the reference FIFO
scripts/mutation_test.py	115	mutation testing of the testbench
scripts/run_synth.py	118	Yosys synthesis of 26 configurations
scripts/run_regression.sh	54	14-run simulation regression

5. Verification Methodology

5.1 Codec unit test

`tb_secded.v` checks that PAR_W is the minimum number of check bits and then, for every data word, decodes the clean codeword, every single-bit corruption and every double-bit corruption. For DATA_W = 8 all 256 data words and all error patterns are exhaustive; for wider words corner patterns (all-zero, all-one, alternating, walking-one) plus random words are used.

5.2 Scoreboard testbench of the FIFO

`tb_sync_fifo_ecc.v` drives inputs on the falling edge and, on every rising edge, advances a transaction-level reference model (a circular queue that remembers each word and the fault mask injected into it). One time-unit later every DUT output is compared with the model: count, full, empty, almost_full, almost_empty, rd_valid, rd_data, sec_err, ded_err, overflow, underflow, sec_cnt and ded_cnt. A word stored with one injected flip must be read back correct with sec_err = 1; with two flips ded_err = 1 is required. With ECC_EN = 0, the model predicts the corrupted word, which measures how many faults reach the output silently.

Test	Purpose
T1	reset values of all outputs
T2	fill to full plus two extra writes: full, almost_full, sticky overflow, data not overwritten
T3	2 x DEPTH cycles of simultaneous read/write while full (write-through)
T4	drain to empty plus extra reads: sticky underflow; read+write while empty
T5	threshold sweep: af_thresh = 0..DEPTH with ae_thresh = DEPTH..0, fill and drain each
T6	single-bit fault injected at every codeword bit position
T7	200 random double-bit faults
T8	synchronous clear of sticky flags and counters
T9	65 540 single-bit events: sec_cnt must saturate at 65 535
T10	constrained random: 6 traffic phases (fill-biased, drain-biased, balanced, 100 % streaming, near-full, mixed), random thresholds, 1.5 % single and 0.5 % double faults, random status clears

5.3 Mutation testing

A passing testbench only means something if it would fail on a wrong design. `scripts/mutation_test.py` copies the RTL, injects one realistic bug (wrong comparator, missing sticky set, wrong pointer enable, broken parity, ...), re-runs the testbench and records whether the bug was detected. The mutation score is the fraction of bugs detected.

6. Results

6.1 Main simulation (DATA_W = 32, DEPTH = 16, ECC_EN = 1)

Metric	Value
Clock cycles simulated	116489
Writes / reads accepted	88337 / 88337
Scoreboard comparisons	1486037
Mismatches	0
Cycles full / empty	12147 / 8790
Maximum occupancy	16 of 16
Write-through cycles at full	2324
Blocked writes / blocked reads	7764 / 7298
Cycles with wr_en and rd_en together	82616
almost_full / almost_empty assertions	462 / 499
Status clears	32
Single / double faults injected	65929 / 182
Codeword bit positions covered	39 of 39
SEC / DED events observed	65929 / 182

Every injected single-bit fault was corrected and every double-bit fault was flagged. In the ECC_EN = 0 configuration with the same kind of traffic, 157 / 120 single/double faults were injected and **277 reads returned corrupted data with no indication** - this is exactly the failure that the SECDED path removes.

6.2 SECDED codec unit test

DATA_W	PAR_W	Codeword	Clean words	Single-bit cases	Double-bit cases	Mismatches
8	4	(13,8)	256	3328	19968	0
16	5	(22,16)	2000	44000	9240	0
32	6	(39,32)	2000	78000	29640	0
64	7	(72,64)	2000	144000	102240	0

6.3 Regression

Run	Result
secded DATA_W=8	PASS
secded DATA_W=16	PASS
secded DATA_W=32	PASS
secded DATA_W=64	PASS
fifo_baseline W32 D16	PASS
fifo_main_w32_d16_ecc (W=32 D=16 ECC=1 +vcd +sat)	PASS
fifo_w8_d4_ecc (W=8 D=4 ECC=1 +seed=11)	PASS
fifo_w16_d8_ecc (W=16 D=8 ECC=1 +seed=22)	PASS
fifo_w64_d64_ecc (W=64 D=64 ECC=1 +seed=33)	PASS
fifo_w32_d16_noecc (W=32 D=16 ECC=0 +seed=44)	PASS

Run	Result
fifo_w32_d16_seed101 (W=32 D=16 ECC=1 +seed=101)	PASS
fifo_w32_d16_seed202 (W=32 D=16 ECC=1 +seed=202)	PASS
fifo_w32_d16_seed303 (W=32 D=16 ECC=1 +seed=303)	PASS
fifo_w32_d16_seed404 (W=32 D=16 ECC=1 +seed=404)	PASS

Total: 14 passed, 0 failed

6.4 Mutation testing

ID	Injected design bug	File	Testbench result
M1	write-through-when-full removed	sync_fifo_ecc.v	KILLED (500 mismatches)
M2	almost_full uses > instead of >=	sync_fifo_ecc.v	KILLED (180 mismatches)
M3	almost_empty uses < instead of <=	sync_fifo_ecc.v	KILLED (208 mismatches)
M4	occupancy ignores reads	sync_fifo_ecc.v	KILLED (500 mismatches)
M5	read pointer advances on rd_en (not rd_fire)	sync_fifo_ecc.v	KILLED (500 mismatches)
M6	overflow flag not sticky-set	sync_fifo_ecc.v	KILLED (500 mismatches)
M7	underflow detection removed	sync_fifo_ecc.v	KILLED (500 mismatches)
M8	status clear does not clear counters	sync_fifo_ecc.v	KILLED (500 mismatches)
M9	SEC counter does not saturate (wraps)	sync_fifo_ecc.v	KILLED (46 mismatches)
M10	full flag one entry early	sync_fifo_ecc.v	KILLED (500 mismatches)
M11	decoder does not correct single errors	secded_dec.v	KILLED (57 mismatches)
M12	encoder overall-parity bit stuck at 0	secded_enc.v	KILLED (500 mismatches)
M13	encoder check bit uses wrong coverage	secded_enc.v	KILLED (500 mismatches)
M14	DED reported as SEC (double errors miscorrected)	secded_dec.v	KILLED (500 mismatches)
M15	rd_valid asserted on rd_en (not rd_fire)	sync_fifo_ecc.v	KILLED (500 mismatches)

Mutation score: 15 / 15 killed (100.0 %). Every injected bug, including subtle ones such as an off-by-one threshold comparison (M2, M3) or a counter that wraps instead of saturating (M9), made the testbench fail. Lint with Verilator -Wall reported zero warnings for every configuration.

6.5 Simulation waveforms

The following waveforms are rendered directly from the VCD file produced by the main simulation (`sim/sync_fifo_ecc.vcd`, viewable in GTKWave with the provided `.gtkw` file).

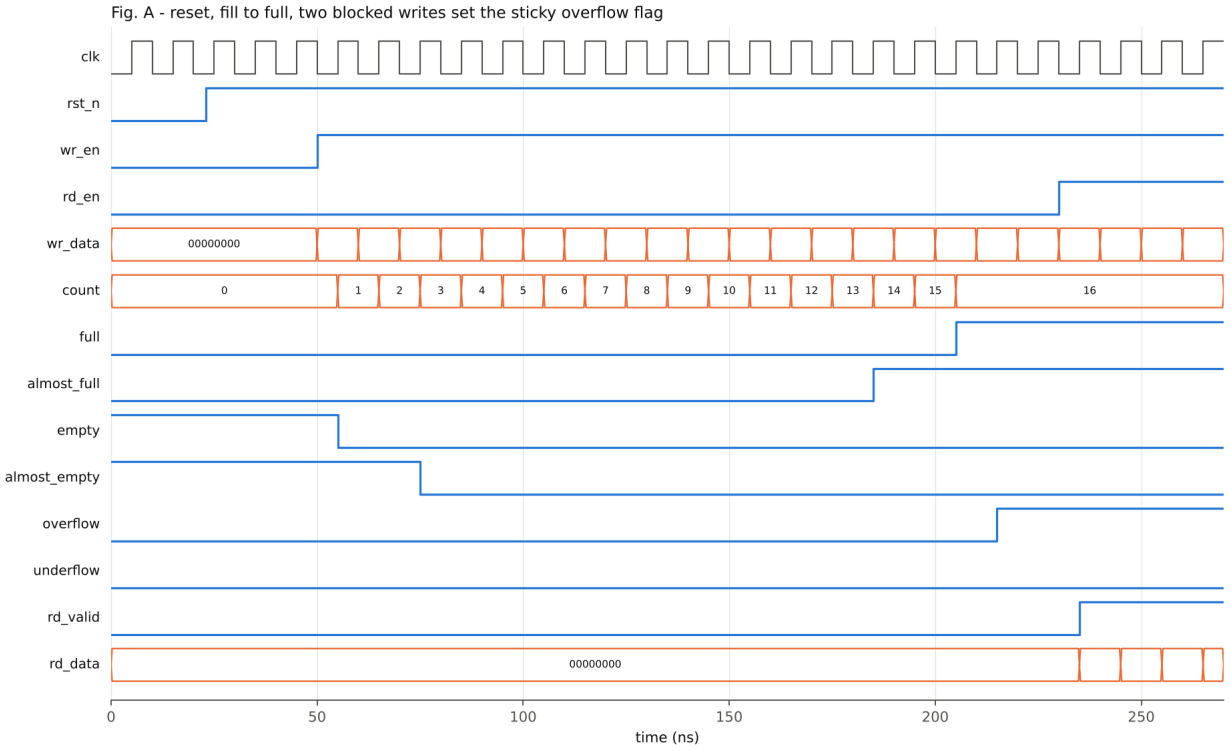


Figure 2 - Reset and fill. count rises 0 to 16; almost_full asserts at 14 (af_thresh = 14), full at 16; the two writes issued while full set overflow.

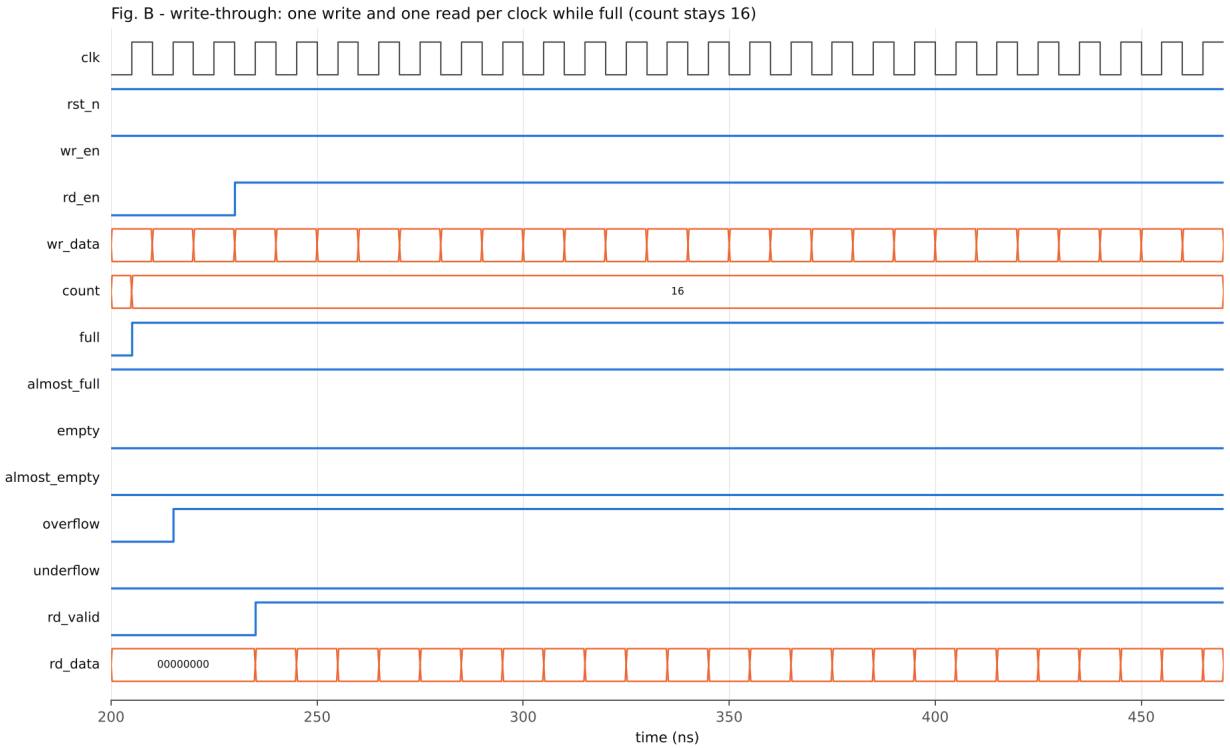


Figure 3 - Write-through: with wr_en and rd_en both high at full, one word enters and one leaves every clock and count stays at 16.

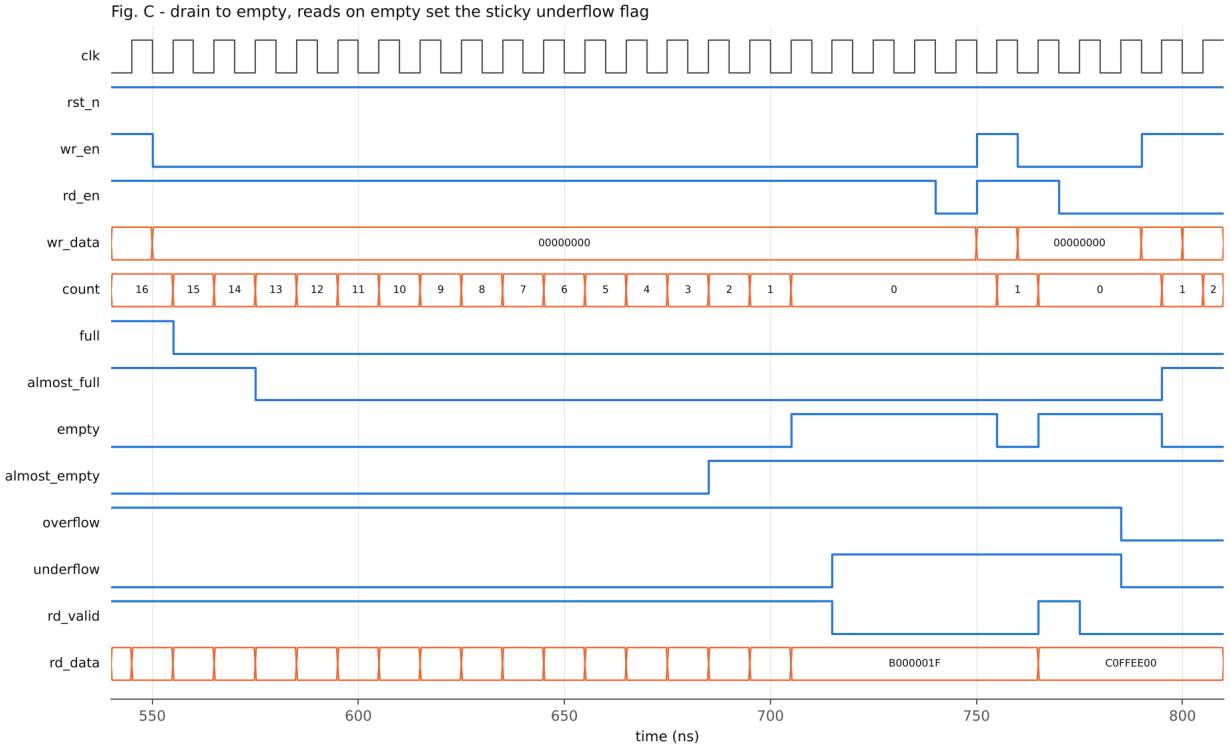


Figure 4 - Drain: reads empty the FIFO; reads while empty are ignored and set the sticky underflow flag.

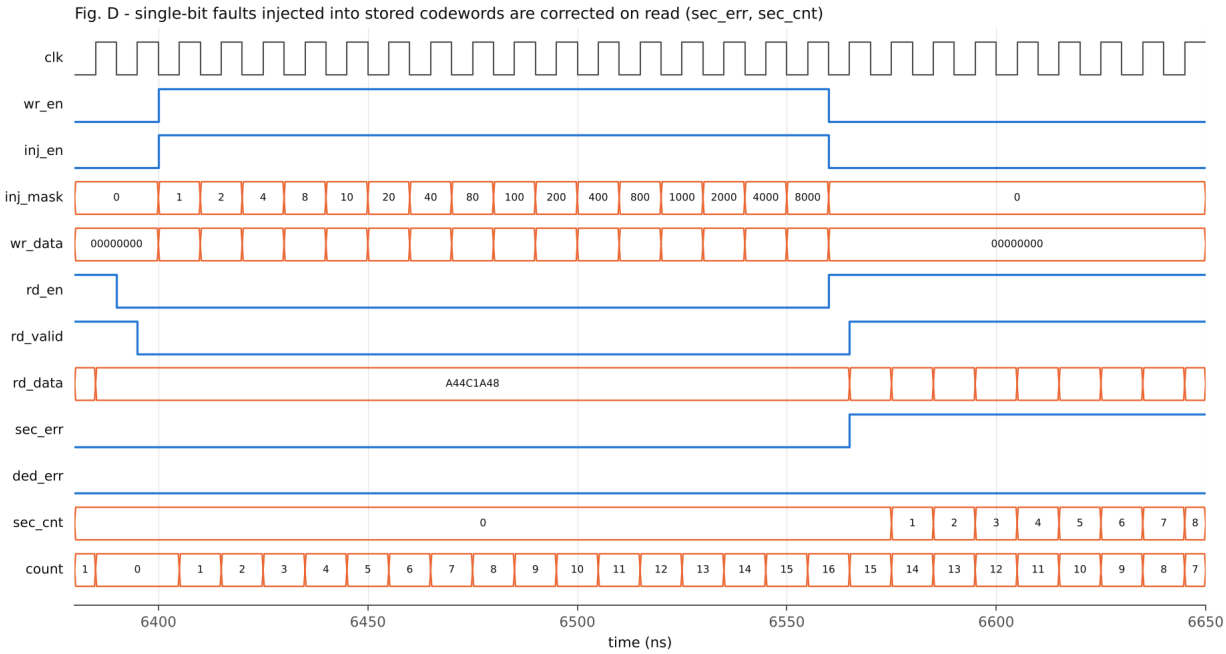


Figure 5 - Fault injection: inj_mask walks a single 1 through the codeword while 16 words are written. On read-back sec_err is high for each word and sec_cnt counts the corrections; the scoreboard confirms rd_data equals the original data.

6.6 Synthesis

All configurations were synthesised with Yosys 0.70, flattening the hierarchy and mapping with ABC to a generic library of two-input gates and 2:1 multiplexers. The storage array is mapped to flip-flops, i.e. the numbers describe a register-file FIFO. Transistor counts use Yosys' `stat -tech cmos` estimate and logic depth is the longest combinational path in gates (`ltp -noff`). No standard-cell library or timing constraints are involved, so the numbers compare architectures, not absolute silicon area or frequency.

Design (DATA_W=32, DEPTH=16)	Flip-flops	Comb. gates	Total cells	CMOS transistors (est.)	Logic depth (gates)
Conventional FIFO (fifo_baseline)	555	539	1094	6156	7
Proposed, ECC_EN = 0	564	662	1226	6972	15
Proposed, ECC_EN = 1 (SECDED)	715	1178	1893	11702	17

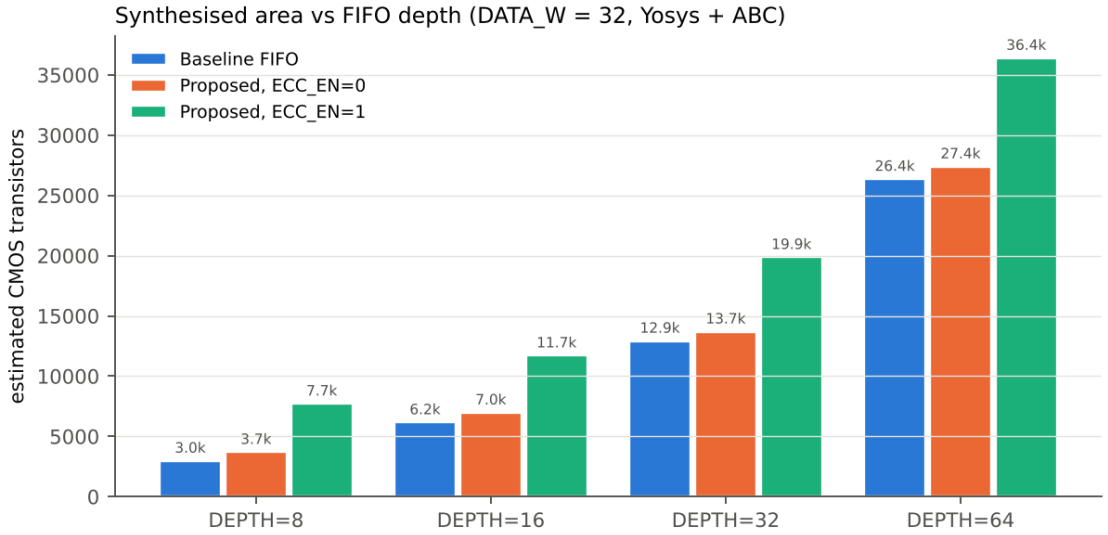


Figure 6 - Estimated transistor count versus depth.

DEPTH	Baseline	Proposed ECC_EN=0	Proposed ECC_EN=1	SECDED overhead vs ECC_EN=0
8	2970	3692	7710	108.8 %
16	6156	6972	11702	67.8 %
32	12920	13688	19872	45.2 %
64	26358	27394	36402	32.9 %

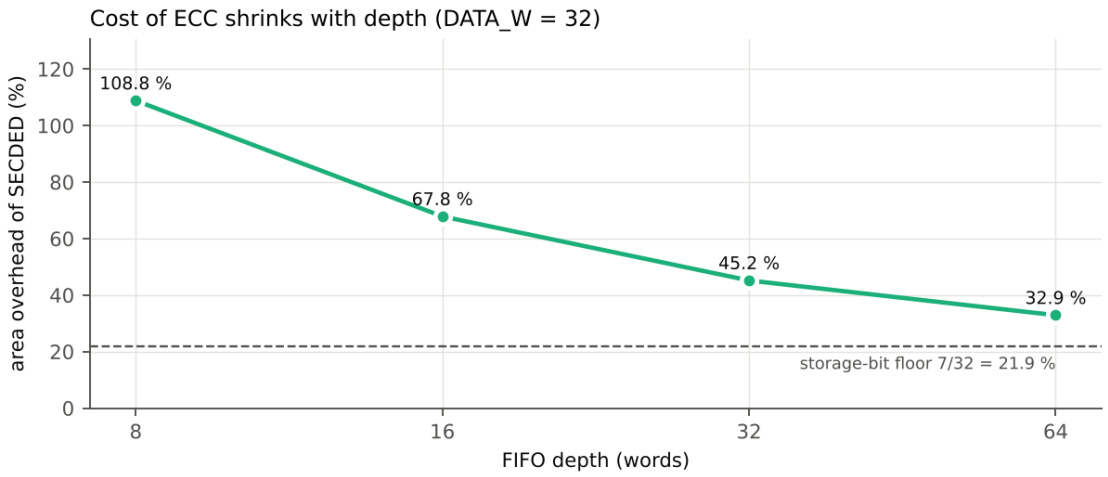


Figure 7 - SECDED overhead approaches the 21.9 % storage floor as depth grows, because the encoder and decoder are shared by all words.

DATA_W	Codeword	Encoder gates	Encoder depth	Decoder gates	Decoder depth
8	(13,8)	15	4	53	7
16	(22,16)	34	6	97	9
32	(39,32)	76	7	173	11
64	(72,64)	175	9	311	10

DATA_W (DEPTH=16)	Baseline transistors	SECDED FIFO transistors	Ratio
8	1836	4538	2.47 x
16	3288	6888	2.09 x
32	6156	11702	1.90 x
64	11916	21080	1.77 x

The proposed control logic without ECC costs 13.3 % more transistors than the baseline at DEPTH = 16 (occupancy counter, programmable comparators, registered flags, error counters). Its logic depth is larger (15 vs 7 gates) because the threshold comparators follow the occupancy adder before the flag registers; this is a deliberate trade for glitch-free registered flags, and the path can be shortened by comparing against count instead of count_nxt at the cost of one cycle of flag latency.

6.7 Reliability benefit (analytical)

Assume each stored bit flips independently with probability p while a word waits in the FIFO. An unprotected 32-bit word is wrong with probability $1-(1-p)^{32} \approx 32p$. With the (39,32) code every single flip is corrected, so a read is wrong only if two or more of the 39 bits flip: $1-(1-p)^{39}-39p(1-p)^{38} \approx 741p^2$. At $p = 10^{-6}$ this is 7.41e-10 versus 3.20e-05, a reduction of about 43,000 times; furthermore, the double-error case is flagged by ded_err instead of passing silently.

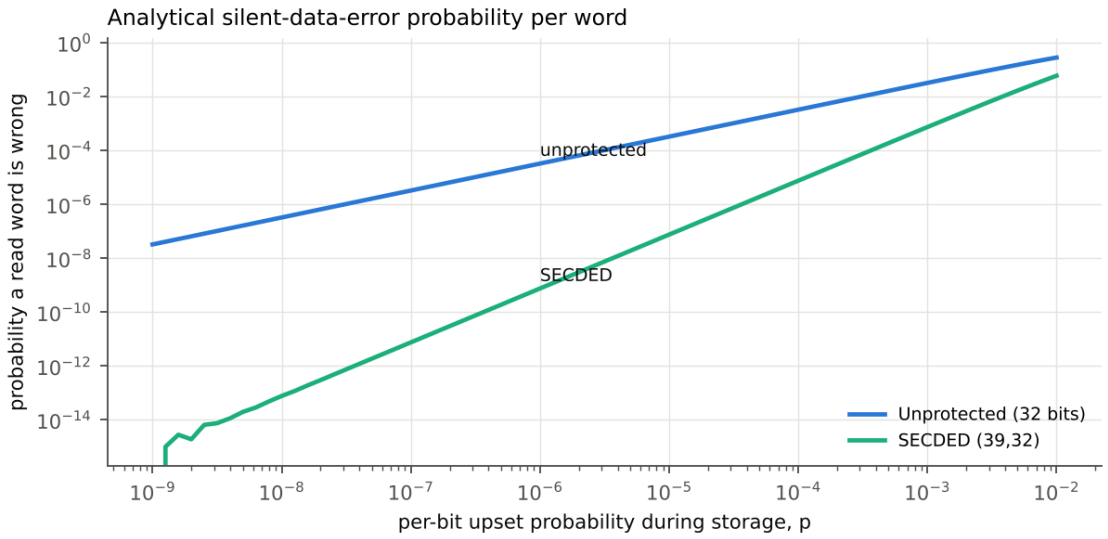


Figure 8 - Probability that a read word is wrong (analytical model).

7. Discussion and Limitations

(a) The decoder is combinational after the read register, so it lies in the path from the read register to `rd_data` (11 gate levels for DATA_W = 32). In a high-frequency design an optional output register can be added, giving a two-cycle read latency. (b) Synthesis used a generic gate library; absolute area and timing require a foundry standard-cell library and, for large depths, an SRAM macro instead of flip-flops. The relative trends reported here are what the generic flow can support. (c) The reliability analysis assumes independent bit flips; multi-cell upsets in adjacent cells are better handled by bit interleaving across words, which is a natural extension. (d) Verification is simulation-based with constrained-random stimulus and functional coverage; formal property checking (e.g. SymbiYosys) would complement it.

8. Conclusion and Future Work

A parameterised SECDED-protected synchronous FIFO was designed in Verilog-2005, verified with a reference-model testbench (1486037 comparisons, 0 mismatches in the main run, 14/14 regression runs passing, all codeword bits fault-injected) whose effectiveness was demonstrated with a 100.0 % mutation score, linted clean with Verilator and synthesised with Yosys. ECC removes all silent single-bit corruption at an overhead that falls from 108.8 % to 32.9 % as the depth grows from 8 to 64 words. Future work: optional output pipeline register, bit-interleaving for multi-cell upsets, background scrubbing of idle entries, formal verification of the flag logic, and an asynchronous (dual-clock) version using Gray-coded pointers.

References

[1] C. E. Cummings, "Simulation and Synthesis Techniques for Asynchronous FIFO Design," SNUG San Jose, 2002.

[2] R. W. Hamming, "Error Detecting and Error Correcting Codes," Bell System Technical Journal, vol. 29, no. 2, pp. 147-160, 1950.

[3] M. Y. Hsiao, "A Class of Optimal Minimum Odd-weight-column SEC-DED Codes," IBM Journal of Research and Development, vol. 14, no. 4, pp. 395-401, 1970.

[4] R. C. Baumann, "Radiation-induced soft errors in advanced semiconductor technologies," IEEE Trans. Device and Materials Reliability, vol. 5, no. 3, pp. 305-316, 2005.

[5] IEEE Std 1364-2005, IEEE Standard for Verilog Hardware Description Language.

[6] C. Wolf, "Yosys Open SYnthesis Suite," https://yosyshq.net/yosys/.

[7] S. Williams, "Icarus Verilog," https://steveicarus.github.io/iverilog/.

[8] Y. Jia and M. Harman, "An Analysis and Survey of the Development of Mutation Testing," IEEE Trans. Software Engineering, vol. 37, no. 5, pp. 649-678, 2011.

Appendix A - How to reproduce

Requirements: Icarus Verilog (iverilog/vvp), Verilator (lint only), Python 3 with matplotlib and reportlab, and Yosys (native binary, or `pip install pyosys`).

```
bash scripts/run_all.sh          # everything below, in order
bash scripts/run_lint.sh         # Verilator -Wall          -> lint/
bash scripts/run_regression.sh   # 14 simulations           -> sim/logs/
python3 scripts/mutation_test.py # 15 mutants               -> sim/logs/mutation_*
python3 scripts/run_synth.py     # 26 synthesis runs        -> syn/
python3 scripts/make_plots.py    # figures                  -> docs/figures/
python3 scripts/make_report.py   # this report              -> docs/Project_Report.pdf
make sim && make wave            # single run + GTKWave
```

Appendix B - Top-level RTL (rtl/sync_fifo_ecc.v)

```
//------------------------------------------------------------------------------
// File        : sync_fifo_ecc.v
// Project     : SECDED-Protected Synchronous FIFO with Programmable Thresholds
// Description : Single-clock FIFO with
//   (1) optional extended-Hamming SECDED protection of the storage array
//       (ECC_EN=1): single-bit upsets are corrected on the fly, double-bit
//       upsets are flagged;
//   (2) run-time programmable almost-full / almost-empty thresholds;
//   (3) registered, glitch-free status flags computed from the NEXT
//       occupancy value (no pointer comparators on the flag outputs);
//   (4) write-through-when-full: a write is accepted while full if a read
//       happens in the same cycle (sustains 100% throughput at full);
//   (5) sticky overflow / underflow flags and saturating SEC / DED event
//       counters with a synchronous clear;
//   (6) a fault-injection port (XOR mask on the stored codeword) so that the
//       ECC path can be verified in simulation and exercised in silicon BIST.
//
// Read latency: rd_data / rd_valid / sec_err / ded_err are valid one clock
// after an accepted rd_en (standard, non-FWFT behaviour).
//------------------------------------------------------------------------------
`timescale 1ns/1ps
module sync_fifo_ecc #(
    parameter integer DATA_W = 32,
    parameter integer DEPTH  = 16,                  // power of 2, >= 2
    parameter integer ECC_EN = 1,                   // 1: SECDED on, 0: off
    parameter integer ADDR_W = $clog2(DEPTH),
    parameter integer PAR_W  = $clog2(DATA_W + $clog2(DATA_W) + 1),
    parameter integer CW_W   = DATA_W + PAR_W + 1
)(
    input  wire              clk,
    input  wire              rst_n,          // asynchronous active-low reset
    // write port
    input  wire              wr_en,
    input  wire [DATA_W-1:0] wr_data,
    // read port
    input  wire              rd_en,
    output wire [DATA_W-1:0] rd_data,
    output reg               rd_valid,
    // status
    output reg               full,
    output reg               empty,
    output reg               almost_full,    // count >= af_thresh
    output reg               almost_empty,   // count <= ae_thresh
    output reg  [ADDR_W:0]   count,
    input  wire [ADDR_W:0]   af_thresh,
    input  wire [ADDR_W:0]   ae_thresh,
    // error reporting
    output reg               overflow,       // sticky
    output reg               underflow,      // sticky
    output wire              sec_err,        // qualified by rd_valid
    output wire              ded_err,        // qualified by rd_valid
    output reg  [15:0]       sec_cnt,        // saturating
```

```
    output reg  [15:0]       ded_cnt,        // saturating
    input  wire              clr_status,
    // fault injection (test only)
    input  wire              inj_en,
    input  wire [CW_W-1:0]   inj_mask
);
    localparam integer MEM_W = (ECC_EN != 0) ? CW_W : DATA_W;
    localparam [ADDR_W:0] DEPTH_V = DEPTH[ADDR_W:0];

    //------------------------------------------------------------------
    // handshake
    //------------------------------------------------------------------
    wire rd_fire = rd_en & ~empty;
    wire wr_fire = wr_en & (~full | rd_en);   // write-through when full

    reg [ADDR_W-1:0] wr_ptr, rd_ptr;

    wire [ADDR_W:0] count_nxt = count + {{ADDR_W{1'b0}}, wr_fire}
                                      - {{ADDR_W{1'b0}}, rd_fire};

    //------------------------------------------------------------------
    // encode + optional fault injection
    //------------------------------------------------------------------
    wire [MEM_W-1:0] wr_word;
    wire [MEM_W-1:0] rd_word;
    reg  [MEM_W-1:0] rd_word_q;
    wire             dec_sec, dec_ded;
    wire [DATA_W-1:0] dec_data;

    generate
        if (ECC_EN != 0) begin : g_ecc
            wire [CW_W-1:0] cw;
            wire [PAR_W-1:0] syn_unused;
            secded_enc #(.DATA_W(DATA_W), .PAR_W(PAR_W), .CW_W(CW_W)) u_enc (
                .data_i (wr_data),
                .cw_o   (cw)
            );
            assign wr_word = cw ^ (inj_en ? inj_mask : {CW_W{1'b0}});

            secded_dec #(.DATA_W(DATA_W), .PAR_W(PAR_W), .CW_W(CW_W)) u_dec (
                .cw_i       (rd_word_q),
                .data_o     (dec_data),
                .sec_o      (dec_sec),
                .ded_o      (dec_ded),
                .syndrome_o (syn_unused)
            );
        end else begin : g_noecc
            assign wr_word  = wr_data ^ (inj_en ? inj_mask[DATA_W-1:0]
                                                : {DATA_W{1'b0}});
            assign dec_data = rd_word_q;
            assign dec_sec  = 1'b0;
            assign dec_ded  = 1'b0;
            // check-bit part of inj_mask has no storage when ECC is off
            wire unused_inj_hi = ^inj_mask;
        end
    endgenerate

    //------------------------------------------------------------------
    // storage array (no reset -> maps to RAM / latch array)
    //------------------------------------------------------------------
    reg [MEM_W-1:0] mem [0:DEPTH-1];

    always @(posedge clk) begin
        if (wr_fire) mem[wr_ptr] <= wr_word;
    end
    assign rd_word = mem[rd_ptr];

    //------------------------------------------------------------------
    // pointers, occupancy, registered flags, read register
    //------------------------------------------------------------------
    always @(posedge clk or negedge rst_n) begin
```

```
        if (!rst_n) begin
            wr_ptr       <= {ADDR_W{1'b0}};
            rd_ptr       <= {ADDR_W{1'b0}};
            count        <= {(ADDR_W+1){1'b0}};
            full         <= 1'b0;
            empty        <= 1'b1;
            almost_full  <= 1'b0;
            almost_empty <= 1'b1;
            rd_valid     <= 1'b0;
            rd_word_q    <= {MEM_W{1'b0}};
        end else begin
            if (wr_fire) wr_ptr <= wr_ptr + 1'b1;
            if (rd_fire) begin
                rd_ptr    <= rd_ptr + 1'b1;
                rd_word_q <= rd_word;
            end
            rd_valid     <= rd_fire;
            count        <= count_nxt;
            full         <= (count_nxt == DEPTH_V);
            empty        <= (count_nxt == {(ADDR_W+1){1'b0}});
            almost_full  <= (count_nxt >= af_thresh);
            almost_empty <= (count_nxt <= ae_thresh);
        end
    end

    //------------------------------------------------------------------
    // error reporting
    //------------------------------------------------------------------
    assign rd_data = dec_data;
    assign sec_err = rd_valid & dec_sec;
    assign ded_err = rd_valid & dec_ded;

    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            overflow  <= 1'b0;
            underflow <= 1'b0;
            sec_cnt   <= 16'd0;
            ded_cnt   <= 16'd0;
        end else if (clr_status) begin
            overflow  <= 1'b0;
            underflow <= 1'b0;
            sec_cnt   <= 16'd0;
            ded_cnt   <= 16'd0;
        end else begin
            if (wr_en & ~wr_fire)             overflow  <= 1'b1;
            if (rd_en &  empty)               underflow <= 1'b1;
            if (sec_err && (sec_cnt != 16'hFFFF)) sec_cnt <= sec_cnt + 1'b1;
            if (ded_err && (ded_cnt != 16'hFFFF)) ded_cnt <= ded_cnt + 1'b1;
        end
    end

endmodule
```

Appendix C - SECDED encoder and decoder

```
//------------------------------------------------------------------------------
// File        : secded_enc.v
// Project     : SECDED-Protected Synchronous FIFO with Programmable Thresholds
// Description : Parameterised extended-Hamming SECDED encoder.
//
//   Codeword layout (CW_W = DATA_W + PAR_W + 1 bits):
//     cw[0]            : overall (even) parity over cw[HAM_W:1]
//     cw[2^j], j<PAR_W : Hamming check bit j
//     cw[other]        : data bits, packed LSB first
//
//   PAR_W is the smallest P with 2^P >= DATA_W + P + 1. The default
//   expression below gives that value for every DATA_W in 1..120
//   (checked exhaustively by tb_secded.v at elaboration of each width used).
//
//   Fully combinational: loops are unrolled at elaboration time.
//------------------------------------------------------------------------------
`timescale 1ns/1ps
module secded_enc #(
    parameter integer DATA_W = 32,
    parameter integer PAR_W  = $clog2(DATA_W + $clog2(DATA_W) + 1),
    parameter integer CW_W   = DATA_W + PAR_W + 1
)(
    input  wire [DATA_W-1:0] data_i,
    output wire [CW_W-1:0]   cw_o
);
    localparam integer HAM_W = DATA_W + PAR_W;   // Hamming positions 1..HAM_W

    reg [HAM_W:0] c;
    integer i, j, k;

    always @* begin
        c = {(HAM_W+1){1'b0}};
        // 1) scatter data bits into the non power-of-two positions
        k = 0;
        for (i = 1; i <= HAM_W; i = i + 1) begin
            if ((i & (i - 1)) != 0) begin
                c[i] = data_i[k];
                k    = k + 1;
            end
        end
        // 2) Hamming check bit j = XOR of every position whose index has bit j set
        for (j = 0; j < PAR_W; j = j + 1) begin
            for (i = 1; i <= HAM_W; i = i + 1) begin
                if ((((i >> j) & 1) == 1) && (i != (1 << j)))
                    c[1 << j] = c[1 << j] ^ c[i];
            end
        end
        // 3) overall parity bit makes the whole codeword even parity
        c[0] = ^c[HAM_W:1];
    end

    assign cw_o = c;

endmodule
```

```
//------------------------------------------------------------------------------
// File        : secded_dec.v
// Project     : SECDED-Protected Synchronous FIFO with Programmable Thresholds
// Description : Parameterised extended-Hamming SECDED decoder (companion of
//               secded_enc.v).
//
//   syndrome == 0, overall parity == 0  -> no error
//   overall parity == 1, syndrome <= HAM_W -> single-bit error, corrected
//                                            (syndrome == 0 -> error was in
//                                             the overall parity bit itself)
//   overall parity == 0, syndrome != 0  -> double-bit error, detected (DED)
//   overall parity == 1, syndrome >  HAM_W -> impossible single error,
//                                            reported as uncorrectable (DED)
//------------------------------------------------------------------------------
```

```
`timescale 1ns/1ps
module secded_dec #(
    parameter integer DATA_W = 32,
    parameter integer PAR_W  = $clog2(DATA_W + $clog2(DATA_W) + 1),
    parameter integer CW_W   = DATA_W + PAR_W + 1
)(
    input  wire [CW_W-1:0]   cw_i,
    output reg  [DATA_W-1:0] data_o,
    output reg               sec_o,       // single error corrected
    output reg               ded_o,       // double (uncorrectable) error
    output reg  [PAR_W-1:0]  syndrome_o
);
    localparam integer HAM_W = DATA_W + PAR_W;

    reg  [HAM_W:0]  fixed;
    reg  [PAR_W-1:0] syn;
    reg              ovp;
    integer i, j, k, syn_i;

    always @* begin
        // syndrome
        syn = {PAR_W{1'b0}};
        for (j = 0; j < PAR_W; j = j + 1)
            for (i = 1; i <= HAM_W; i = i + 1)
                if (((i >> j) & 1) == 1)
                    syn[j] = syn[j] ^ cw_i[i];
        ovp   = ^cw_i;
        syn_i = {{(32-PAR_W){1'b0}}, syn};

        fixed = cw_i;
        sec_o = 1'b0;
        ded_o = 1'b0;

        if (ovp) begin
            if (syn_i <= HAM_W) begin
                sec_o = 1'b1;
                for (i = 0; i <= HAM_W; i = i + 1)
                    if (syn_i == i) fixed[i] = ~fixed[i];
            end else begin
                ded_o = 1'b1;
            end
        end else if (syn != {PAR_W{1'b0}}) begin
            ded_o = 1'b1;
        end

        // gather data bits
        data_o = {DATA_W{1'b0}};
        k = 0;
        for (i = 1; i <= HAM_W; i = i + 1) begin
            if ((i & (i - 1)) != 0) begin
                data_o[k] = fixed[i];
                k         = k + 1;
            end
        end
        syndrome_o = syn;
    end

endmodule
```